



7220240891-KT

西南科技大学

硕士研究生学位论文开题报告

学 号: 7220240891

姓 名: 王娜

学 院: 计算机科学与技术学院

专业(领域): 软件工程

研究方向: 软件工程(全日制/非全日制)

导师姓名: 张晓蓉

攻读学位:

2025 年 12 月 2 日

论文题目	基于 LLM 多智能体协作的单元测试代码自动生成方法研究		
选题来源	其他	论文类型	应用研究
开题日期	2025-12-2	开题地点	东六 E 学术报告厅

一、立题依据（包括研究目的、意义、国内外研究现状和发展趋势，需结合科学研究发展趋势来论述科学意义；或结合国民经济和社会发展中迫切需要解决的关键科技问题来论述其应用前景。附主要参考文献目录）（不少于 800 字）

随着软件开发迭代速度的加快和软件系统复杂度的不断提升，单元测试作为确保软件质量的关键环节，其重要性日益显著。单元测试通过对软件最小可独立执行单元（如函数、方法和类）进行早期验证，能够在编码阶段有效识别代码中逻辑错误（如条件判断疏漏、循环边界处理异常）和边界异常情况（如输入空值、非法参数、极值数据），显著降低了后期缺陷修复成本，并为后续的集成测试、系统测试等环节提供坚实基础。传统的单元测试方法主要依赖开发人员手工编写测试代码。这一过程要求开发人员对业务逻辑有深入透彻的理解，还需具备系统化的测试思维，能够针对每一个函数、类或模块，细致地编写完整测试代码。然而，这种手工编写模式存在显著劣势。一方面，手工编写测试代码是一项繁重且耗时的工作，据行业调研数据显示，在代码量超 50 万行的中大型软件项目中，手工编写单元测试代码的工作量通常占据开发总工作量的 30%至 50%，部分涉及复杂算法的模块测试代码编写时间甚至超过业务代码开发时间，且非常容易因为人为疏忽导致测试覆盖不全面；另一方面，随着软件系统的迭代升级，业务逻辑的调整会直接导致原有测试代码失效，开发人员需同步修改甚至重写测试代码，测试成本随之线性增长，严重制约了软件开发的效率。为解决传统手工模式的痛点，自动化单元测试技术应运而生，其核心逻辑是通过程序脚本或专用工具直接产出可编译运行的测试代码、执行测试生成与驱动执行的全流程，无需人工干预，从而大幅提升测试效率。

早期的自动化单元测试生成技术研究主要聚焦于基于搜索、随机生成和符号执行三大方向。其中，基于搜索的技术通过定义目标函数（如代码覆盖度），运用遗传算法、模拟退火等搜索策略在测试空间中迭代优化，系统性地探索测试输入空间，有效提升测试覆盖率；随机生成技术依据预设的数据分布规则随机生成测试数据，凭借简单直接的实现方式，可快速生成大量测试，在早期测试阶段能高效发现明显缺陷；符号执行技术使用符号值代替具体数据执行程序，记录路径约束条件并通过求解约束生成测试，这种方式通过深入探索程序的不同执行路径，精准定位复杂逻辑中的潜在问题。尽管早期自动化单元测试生成技术在一定程度上解决了手工模式的效率问题，但在面对当前高复杂度软件系统的测试需求（如嵌套分支结构、动态依赖关系）时，基于搜索的技术易陷入局部最优，随机生成技术难以命中有效路径，符号执行技术则因约束爆炸导致效率骤降。最终这些方法都面临着测试覆盖度不足、生成的测试可读性差、难以满足日益严苛的高复杂度软件测试需求。

最近，大型语言模型（Large Language Models, LLM）在单元测试生成中的应用在学术界得到了广泛的探索。当前，基于 LLM 的自动化单元测试生成研究通常采用固定提示模板来指导 LLM 生成测试代码。通过预定义的结构方法提取待测试项目中的关键信息，例如函数名称、参数类型等，并将其作为测试对象，随后生成提示指导 LLM 输出测试。尽管现有的 LLM 驱动自动化单元测试生成的方法能够较好地解决早期自动化单元测试生成技术所面临的局限，如测试覆盖度不足、测试代码可读性差以及难以满足测试需求，但

仍然存在以下问题：

(1)**信息静态**：一方面体现为提示构建的静态化，因固定提示的反复使用、缺乏对测试任务的个性化描述，且简单的提示填充方式易引发上下文溢出，导致 LLM 生成过程频繁受限、出现中止或停滞，还极易产生幻觉并持续输出无效或重复内容；另一方面表现为焦点提取的静态性，现有焦点提取方式仅能获取静态焦点信息，使得构建的提示内容极为单一，无法让 LLM 充分理解测试意图，进而造成生成质量不佳或修复幻觉的问题。

(2)**验证粗放**：在 LLM 驱动自动化单元测试生成后往往采用系统验证器去检查单元测试是否存在编译运行错误。然而有一些测试类或测试方法是因为无关或无用的错误才导致无法编译运行，这些错误实际上对整个测试并无影响，但是单一的系统验证机制会直接丢弃该测试类或测试方法，导致测试类或测试方法数量减少，进而降低整体的测试覆盖率。

(3)**半自动化**：当前，LLM 驱动单元测试自动生成过程中，有时仍需依赖人工介入。如 LLM 输出测试代码后，需要先手动修正测试代码的格式错误，然后手动创建测试类，交给编译验证器去检验，此时如果再出现任何编译错误，还需要手动去修正。这种半自动化模式未对可自动化环节做分层处理，仍强制人工全程参与，导致人工投入的时间成本大幅增加，进而降低整体测试生成的流程效率。

尽管存在上述问题，LLM 在自动化单元测试生成领域的潜力仍然不容忽视。要充分发挥 LLM 的优势就必须克服以上问题，必须重视全自动化的重要性，必须寻求新的方法来提升 LLM 对测试意图的理解以及生成测试代码的覆盖率。

参考文献

- [1].Sen K .DART: directed automated random testing[C]//International Haifa Verification Conference on Hardware & Software: Verification & Testing.ACMPUB27New York, NY, USA, 2005.DOI:10.1145/1064978.1065036.
- [2].Cadar C , Dunbar D , Engler D R .KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs[C]. Usenix Conference on Operating Systems Design & Implementation.Usenix Association, 2009.DOI:10.1109/ROBIO.2013.6739715.
- [3].Fraser G , Arcuri A .EvoSuite: Automatic test suite generation for object-oriented software[J].ACM, 2011.DOI:10.1145/2025113.2025179.
- [4].José Miguel Rojas, Vivanti M , Arcuri A ,et al.A detailed investigation of the effectiveness of whole test suite generation[J].Empirical Software Engineering, 2016, 22(2):1-42.DOI:10.1007/s10664-015-9424-2.
- [5].Deepthi A .A Survey on Readability Improving Techniques on Automated Unit Test Case Generation[J].International journal of engineering research and technology, 2018, 4.
- [6].Wang S , Shrestha N , Subburaman A K ,et al.Automatic Unit Test Generation for Machine Learning Libraries: How Far Are We?[J].ACM, 2021.DOI:10.1109/ICSE43902.2021.00138.
- [7].Liu X , Yu P .Randoop-TSR: Random-based Test Generator with Test Suite Reduction[J].Proceedings of the 13th Asia-Pacific Symposium on Internetwork, 2022.DOI:10.1145/3545258.3545280.
- [8].Lukasczyk S , Fraser G .Pynguin: Automated Unit Test Generation for Python[J]. 2022.DOI:10.48550/arXiv.2202.05218.
- [9].Bhatia S , Gandhi T , Kumar D ,et al.Unit Test Generation using Generative AI : A Comparative Performance Analysis of Autogeneration Tools[J].IEEE, 2023.
- [10].Chen X , Lin M , Schrli N ,et al.Teaching Large Language Models to Self-Debug[J]. 2023.
- [11].Chen Y , Hu Z , Zhi C ,et al.ChatUniTest: A Framework for LLM-Based Test Generation[J]. 2023.
- [12].Dakhel A M , Nikanjam A , Majdinasab V ,et al.Effective Test Generation Using Pre-trained Large Language Models and Mutation Testing[J]. 2023.
- [13].Dias T , Batista A , Maia E ,et al.TestLab: An Intelligent Automated Software Testing Framework[J]. 2023.
- [14].Fontes A , Gay G .The integration of machine learning into automated test generation: A systematic mapping study[J].Software Testing: Verification & Reliability, 2023, 33(4).DOI:10.1002/stvr.1845.

- [15].Ouédraogo, Wendkuni C, Plein L ,Kaboré, Kader,et al.Enriching Automatic Test Case Generation by Extracting Relevant Test Inputs from Bug Reports[J]. 2023.
- [16].Paul R , Hossain M M , Siddiq M L ,et al.Enhancing Automated Program Repair through Fine-tuning and Prompt Engineering[J]. 2023.
- [17].Pryzant R , Iter D , Li J ,et alAutomatic Prompt Optimization with "Gradient Descent" and Beam Search[J]. 2023.
- [18].Rho S , Martens P , Shin S ,et al.Coyote C++: An Industrial-Strength Fully Automated Unit Testing Tool[J]. 2023.
- [19].Zhang Y , Li G , Jin Z ,et al.Neural Program Repair with Program Dependence Analysis and Effective Filter Mechanism[J]. 2023.
- [20].Alshahwan N , Chheda J , Finegenova A ,et al.Automated Unit Test Improvement using Large Language Models at Meta[J]. 2024.
- [21].Alagarsamy S , Tantithamthavorn C , Aleti A .A3Test: Assertion-Augmented Automated Test case generation[J].Information and Software Technology, 2024, 176(000).DOI:10.1016/j.infsof.2024.107565.
- [22].He Y , Huang J , Rong Y ,et al.UniTSyn: A Large-Scale Dataset Capable of Enhancing the Prowess of Large Language Models for Program Testing[J]. 2024.
- [23].Guo D , Zhu Q , Yang D ,et al.DeepSeek-Coder: When the Large Language Model Meets Programming -- The Rise of Code Intelligence[J]. 2024.
- [24].Gu S , Fang C , Zhang Q ,et al.Improving LLM-based Unit test generation via Template-based Repair[J]. 2024.
- [25].Gu S , Zhang Q , Li K ,et al.TestART: Improving LLM-based Unit Testing via Co-evolution of Automated Generation and Repair Iteration[J]. 2024.
- [26].Lops A , Narducci F , Ragone A ,et al.A System for Automated Unit Test Generation Using Large Language Models and Assessment of Generated Test Suites[J]. 2024.DOI:10.1109/ICSTW64639.2025.10962454.
- [27].Nunez A , Islam N T , Jha S K ,et al.AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing[J]. 2024.
- [28].Ouedraogo W C , Kabore K , Song Y ,et al.LLMs and Prompting for Unit Test Generation: A Large-Scale Evaluation[C]. 2024 39th IEEE/ACM International Conference on Automated Software Engineering (ASE).0[2025-10-01].
- [29].Parasaram N , Yan H , Yang B ,et al.The Fact Selection Problem in LLM-Based Program Repair[C]. 2024.DOI:10.1109/ICSE55347.2025.00162.
- [30].Sapozhnikov A , Olsthoorn M , Panichella A ,et al.TestSpark: IntelliJ IDEA's Ultimate Test

- Generation Companion[J]. 2024.
- [31].Shang Y , Zhang Q , Fang C ,et al.A Large-scale Empirical Study on Fine-tuning Large Language Models for Unit Testing[J]. 2024.
- [32].Sherifi B , Slhoub K , Nembhard F .The Potential of LLMs in Automating Software Testing: From Generation to Reporting[J]. 2024.
- [33].Steenhoek B , Tufano M , Sundaresan N ,et al.Reinforcement Learning from Automatic Feedback for High-Quality Unit Test Generation[C]. 2024.DOI:10.1109/DeepTest66595.2025.00011.
- [34].Wang Z , Liu K , Li G ,et al.HITS: High-coverage LLM-based Unit Test Generation via Method Slicing[J]. 2024.
- [35].Yang L , Yang C , Gao S ,et al.On the Evaluation of Large Language Models in Unit Test Generation[J]. 2024.
- [36].Zhang Q , Fang C , Xie Y ,et al.A Systematic Literature Review on Large Language Models for Automated Program Repair[J]. 2024.
- [37].Zhang Z , Wang Y , Wang C ,et al.LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation[J]. 2024.
- [38].Broide L , Stern R .EvoGPT: Enhancing Test Suite Robustness via LLM-Based Generation and Genetic Optimization[J]. 2025.
- [39].Chen Y , Li Z , Yang C ,et al.Large Language Models are Few-shot Multivariate Time Series Classifiers[J]. 2025.DOI:10.1007/s10618-025-01145-z.
- [40].Guo X , Li C , Tsuchiya T .Boundary Value Test Input Generation Using Prompt Engineering with LLMs: Fault Detection and Coverage Analysis[J]. 2025.
- [41].Khelladi D E , Reux C , Acher M .Unify and Triumph: Polyglot, Diverse, and Self-Consistent Generation of Unit Tests with LLMs[J]. 2025.
- [42].Konstantinou M , Degiovanni R , Zhang J M ,et al.YATE: The Role of Test Repair in LLM-Based Unit Test Generation[J]. 2025.
- [43].Wang Y , Xia C , Zhao W ,et al.ProjectTest: A Project-level LLM Unit Test Generation Benchmark and Impact of Error Fixing Mechanisms[J]. 2025.
- [44].Yaraghi A S , Holden D , Kahani N ,et al.Automated Test Case Repair Using Language Models[J].IEEE Transactions on Software Engineering, 2025(4):51.DOI:10.1109/tse.2025.3541166.
- [45].Zhang Y , Lu Q , Liu K ,et al.CITYWALK: Enhancing LLM-Based C++ Unit Test Generation via Project-Dependency Awareness and Language-Specific Knowledge[J]. 2025.

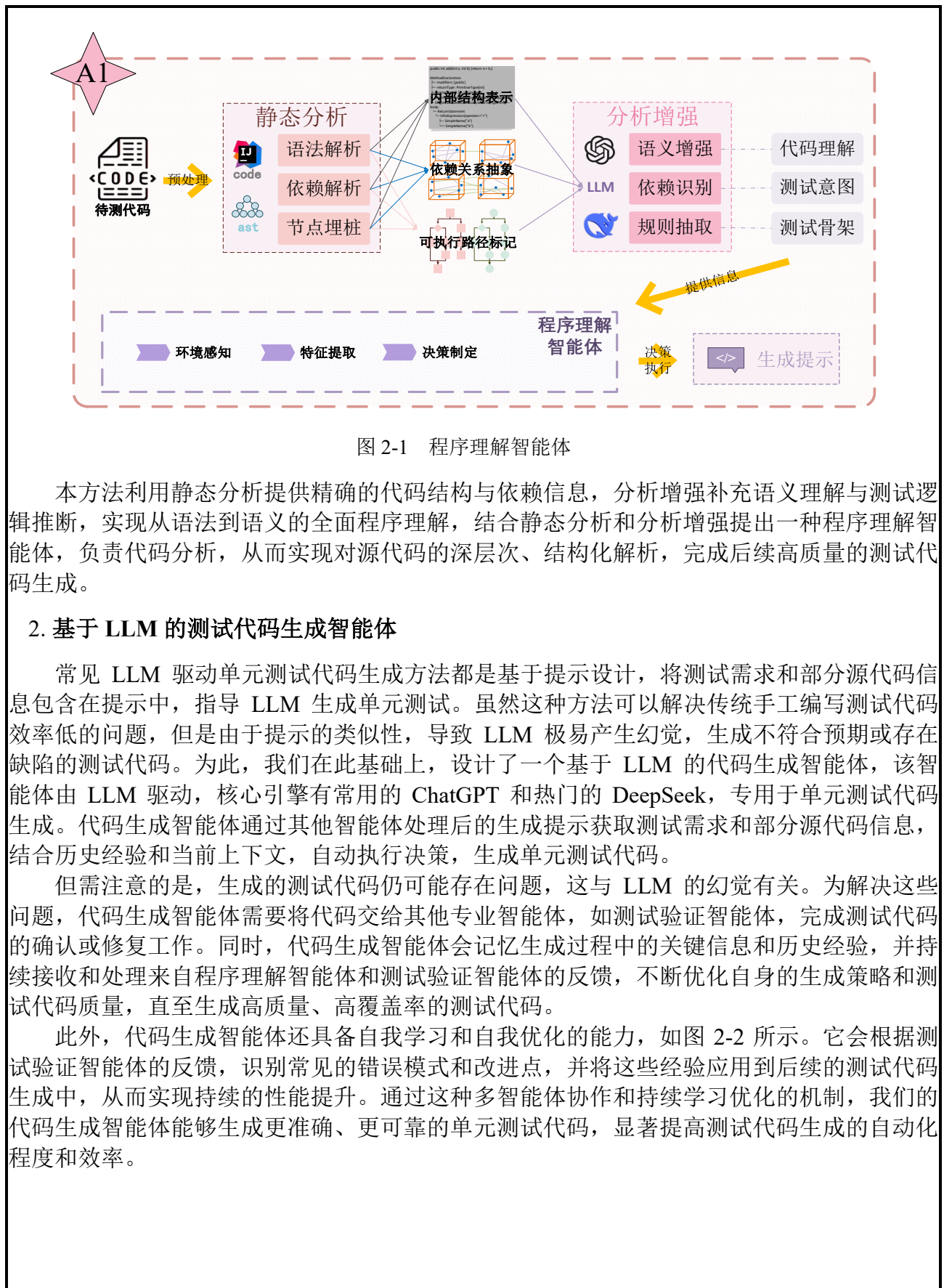
二、研究内容和目标（说明课题的具体研究内容，研究目标和效果，以及拟解决的关键科学问题。此部分为重点阐述内容）（不少于 2500 字）

本研究旨在探索一种更为全面、细致的单元测试代码生成方法，该方法通过 LLM 智能体开发与单元测试集成两大概念出发，通过赋予 LLM 主动参与的能力以生成更精准的测试摘要，推动其从被动的代码生成角色向主动的任务参与角色转变；同时围绕具体测试任务，整合智能体的自动编码、逻辑推理、记忆迭代等核心特性，使其能够独立或协作完成复杂的代码相关任务。与纯 LLM 驱动测试生成方法不同，本方法以 LLM 为基础，将单元测试代码生成过程深度嵌入智能体架构中，并保留生成过程中存在缺陷但具备潜在价值的测试代码，最终实现高质量、高覆盖率单元测试代码的自动化生成。为支撑这一方法的构建与验证，本研究针对自动化单元测试技术与 LLM 智能体的技术原理及应用现状展开了系统性的研究与分析。

1. 基于静态分析和分析增强的程序理解智能体

静态分析方法（如基于抽象语法树（AST）或深度学习方法的技术）通过分析源代码而无需执行程序来识别问题。然而，这些方法往往难以捕捉隐含的业务语义和复杂的上下文依赖，且可能遗漏跨文件、跨模块的动态行为问题。相比之下，利用 LLM 来分析待测试源代码具有强大的自然语言理解和上下文建模能力，能够自动推断测试意图、识别潜在边界条件并生成高可读性的测试骨架，但存在对代码结构理解不精确、可能产生幻觉或不符合语法规范的问题。因此，静态分析与 LLM 增强分析方法对于全面的代码理解至关重要，因为静态分析能提供精确的语法结构和依赖关系，而 LLM 增强分析方法则能补充语义层面的洞察和测试意图的自动生成。

首先，通过静态分析技术解析源代码，构建 AST，识别被测方法的签名、参数类型及依赖关系；通过依赖解析与节点关系抽取建立内部结构、依赖关系的抽象表达。同时，为了捕捉潜在的可执行路径与分支逻辑，在静态分析过程中对 AST 进行节点插桩，标记可执行路径与关键控制节点，从而构建程序的控制流与数据流信息，辅助后续测试逻辑生成。之后将这些信息结构化地同步交给 LLM，LLM 在此基础上自主对程序源代码进行进一步全方位深度分析，包括但不限于语法结构、依赖关系和方法内容，帮助 LLM 自动识别并提取程序源代码中的关键要素，如方法签名、参数类型、功能模块等，帮助 LLM 理解程序源代码的代码内容、测试意图，以输出高可读性、高覆盖性的测试骨架。之后这些信息将被系统性地处理和组织交付给程序理解智能体，由其进行语义增强、测试意图识别与生成提示构建，并最终输出生成提示，从而为代码生成智能体提供明确而充分的指导信息，如图 2-1 所示。



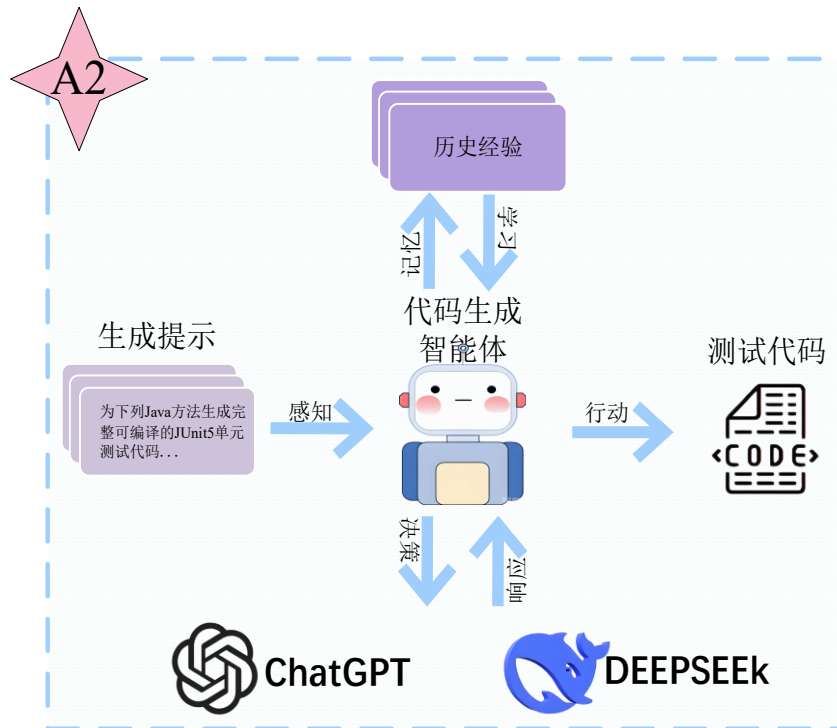


图 2-2 代码生成智能体

3. 基于动态验证和目标修复的测试验证智能体

传统编译验证器全部采用“一刀切”式，一次性把整个测试类送进编译器，一旦出现任何错误，系统就会终止验证。然而在面临一些无影响错误时，编译器仍会直接丢弃整个测试类，导致可行性测试数量减少，进而降低测试覆盖率。

我们的方法利用 LLM 强大的自然语言处理和代码理解能力，通过设计全面而细致的验证标准去指导 LLM 参与测试验证过程，实现对测试代码的全面验证。该验证标准体系涵盖静态语法规则验证、动态执行逻辑验证和编译运行可行性验证三部分内容。其中，静态语法规则验证重点关注测试代码的语法正确性，确保其没有语义错误，例如是否正确导入了所需的包路径、类型是否匹配、变量名是否一致等；动态执行逻辑验证通过模拟测试运行环境，检查测试代码的逻辑是否完整且符合预期，特别关注其是否能够覆盖正常流程、边界条件以及异常处理等关键场景；编译运行可行性验证则将重点检查测试代码的可编译性和可执行性，确保其能够在目标环境中顺利运行，避免因环境不匹配导致的测试失败。当验证发现错误时，测试验证智能体将自动从验证结果中提取错误报告，如图 2-3 所示。此时，由于智能体的智能决策特性，错误报告中自动包含对应的修复行动，智能体自动根据行为编号与历史记忆中已有的修复策略，进行选择。我们修复策略能够依据错误的性质和影响范围，选择最合适的修复策略，如图 2-4 所示。例如，对于仅涉及单行代码的错误，智能体会选择行级修复策略；而对于影响整个类结构或跨方法的错误，则会选择类级修复策略。这种精细化的修复策略选择，不仅能够提高修复的准确性，还能减少不必要的代码改动，从而保证测试代码的质量和稳定性。修复完成后，测试验证智能体基于修复内容将同步更新验证策略，即在初始验证标准上动态调整针对该类错误的验证规则，避免过度验证。同时测试验证智能体自主感知错误性质和验证规则，将其归类记忆并反馈给其他智能体。

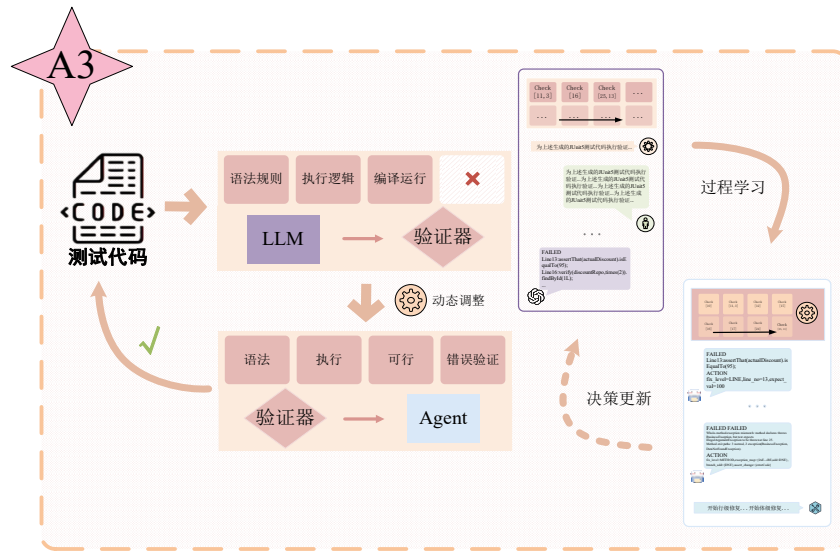


图 2-3 测试验证智能体-动态验证

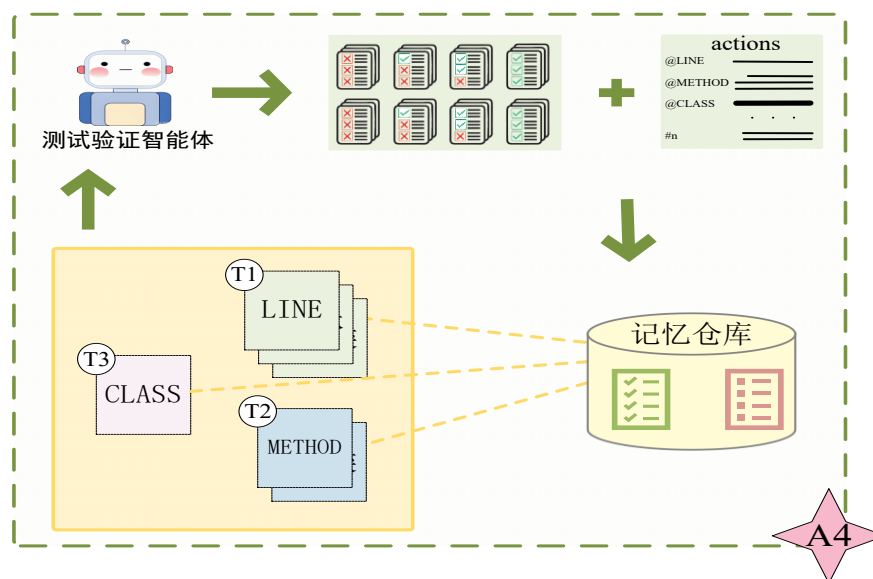


图 2-4 测试验证智能体-目标修复

测试验证智能体不仅能确保测试代码符合语法规则、测试逻辑符合预期、最终测试可执行编译且没有语法语义错误，还能根据历史修复经验动态调整验证规则，解决传统方法验证粗放的潜在问题，对于保障生成测试质量和可行具有重大意义。

4. 拟达到效果和拟解决问题

本研究以基于 LLM 的多智能体协作为研究切入点，提出让 LLM 深度参与单元测试全过程，进一步实现单元测试代码生成任务全过程智能化，拟达到强化 LLM 对测试意图的深度理解，拟实现提升测试代码的有效性与准确性。

4.1 解决测试意图不明、信息静态的问题

在传统的单元测试实践中，测试意图往往不够明确，导致生成的测试代码无法充分覆盖

实际的测试需求。此外，测试信息的静态性限制了测试代码对动态变化的适应能力。本研究通过引入基于 LLM 的多智能体协作，旨在解决这些问题。LLM 的自然语言处理能力使其能够深入理解测试需求的文本描述，从而明确测试意图。同时，多智能体系统能够动态调整测试策略，以适应不断变化的测试环境和需求。程序理解智能体负责解析代码和提取关键信息，而代码生成智能体则根据这些信息和明确的测试意图来自动生成测试代码。这样，测试代码不仅能够准确反映测试需求，还能够适应需求的变化，提高测试的有效性和准确性。

4.2 解决测试验证粗放、修复工程庞大的问题

现有的测试验证方法往往采用粗放的验证方式，一旦测试代码中存在错误，就可能需要人工介入进行修复，这不仅效率低下，而且容易引入新的错误。本研究提出的多智能体协作方法，通过动态验证和目标修复策略，能够精确地识别和定位测试代码中的问题，并自动生成修复方案。测试验证智能体在验证过程中不仅能够发现错误，还能够分析错误的原因，并与代码生成智能体协作，自动应用修复策略。这种精细化的验证和修复过程减少了人工干预的需求，降低了修复工程的复杂性，提高了测试代码的质量和可靠性。

4.3 解决单元测试代码生成半自动化的问题

尽管现有的 LLM 驱动的单元测试生成方法在一定程度上实现了自动化，但仍存在许多需要人工介入的环节，如格式修正、依赖管理等，这限制了自动化的深度和广度。本研究提出的多智能体协作框架，通过程序理解智能体、代码生成智能体和测试验证智能体的紧密合作，实现了单元测试代码生成的全过程自动化。程序理解智能体负责理解代码和测试需求，代码生成智能体负责生成测试代码，而测试验证智能体负责验证和优化这些代码。这种全流程的自动化不仅提高了测试代码生成的效率，还确保了测试代码的质量，从而实现了单元测试代码生成的真正自动化，减少了人工干预，提高了测试流程的整体效率。

三、研究方案设计及可行性分析（包括：研究方法，技术路线，理论分析、计算、实验方法和步骤及其可行性等）（不少于 800 字）

1. 数据集预处理

我们选择 Methods2Test 和 Defects4J 这两个数据集作为研究对象。Methods2Test 是一个面向方法级监督学习的 Java 单元测试数据集，由微软研究院 Tufano 等人于 2020 年发布并持续维护，专门用于单元测试自动生成任务。它从 9410 个不同的 GitHub 开源仓库中抽取了 78 万余条平行句对，每条样本包含一个<被测方法，对应真实@Test 方法>的完整映射，并额外给出被测类的构造函数、公共方法与公共字段等上下文。而 Defects4J 是一个以项目级真实缺陷为核心的 Java 实验基准，由华盛顿大学 Michael Ernst 团队 2014 年建设，共收录 17 个开源项目。这两个数据集在学术界被广泛使用，尤其在单元测试研究领域。其中 Methods2Test 数据集侧重于单个方法，能够为单元测试代码生成研究提供细粒度的实验对象。而 Defects4J 数据集则关注整个项目，能够为单元测试代码生成研究提供更贴近真实软件开发的实验场景。如表 1-1 所示。

表 1-1 数据集介绍

数据集名称	发布机构与年份	数据规模与来源	研究侧重点
Methods2Test	微软研究院，2020 年	从 9410 个 GitHub 开源仓库抽取	单个方法
Defects4J	华盛顿大学，2014 年	共收录 17 个开源项目	整个项目

在正式开展实验之前，我们需要充分考虑智能体在解析代码逻辑结构时可能面临的巨大负担。代码中的空白行和各类注释虽然对于人类开发者理解代码有着重要的辅助作用，但对于智能体而言，它们往往会增加不必要的处理复杂度。

因此，我们需要采取一系列的预处理措施。首先，我们需要删除源代码中的所有空白行，这些空白行在代码文件中分布较为零散，有些是为了代码排版的美观而添加，有些则是在代码编辑过程中遗留下来的。删除后再逐行检查代码文件，确保没有任何一个空白行被遗漏。

然后，我们需要针对各类注释，包括函数注释、类注释和行注释等，进行全面的清理工作。为此，我们选择专门的脚本工具，识别并移除这些注释内容，以减少非必要代码行数和字符数对智能体的干扰，使得智能体能够将更多的计算资源集中在核心代码逻辑的解析上。

最后，为了进一步提高实验数据的质量和有效性，我们还需要手动剔除代码中的冗余代码片段。冗余代码片段是指那些在程序中虽然存在，但对程序的最终功能和输出结果没有实质性影响的代码部分。它们可能是由于代码开发过程中的重复编写、未使用的变量定义或者无效的逻辑分支等原因产生的。这样做不仅可以确保实验数据的简洁性，避免智能体在处理这些无用代码时浪费计算资源，还可以保证实验数据能够准确地反映程序的核心功能和逻辑。

2. 单元测试全过程划分

通过对近 3 年 50 余篇 LLM 在自动化单元测试领域的研究文献进行系统性梳理和分析，我们单元测试代码生成的全部过程细致拆分为程序理解、测试生成、测试验证以及修

复错误，让 LLM 能够深度参与单元测试全过程并在每一个关键步骤发挥作用，实现多智能体协作的自动化单元测试代码生成方法，如图 3-1 所示。

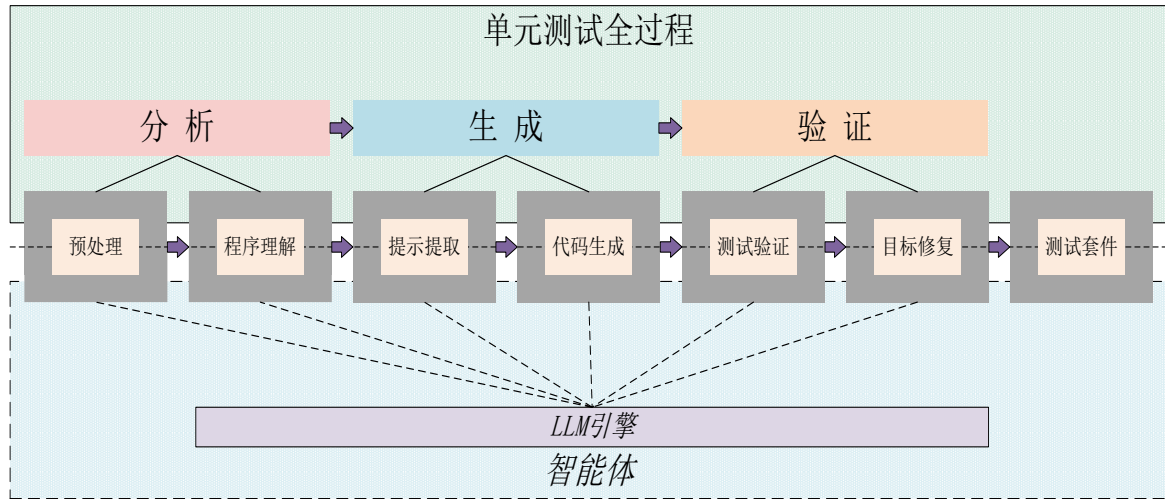


图 3-1 单元测试全过程划分

2.1 程序理解

在 LLM 驱动自动化单元测试生成的研究中，程序理解是第一步也是至关重要的一步，不仅涉及对程序源代码的深度解析，以使 LLM 能够有效掌握程序的结构、功能和行为，而且奠定了后续测试生成的基础，在很大程度上决定最终生成测试的质量和有效性。让 LLM 参与程序理解过程，目的是为确保生成的测试代码能够精确地映射出待测源代码的真实意图，基于 LLM 设计智能体让其自主对程序源代码进行全方位深度分析，包括但不限于语法结构、依赖关系和方法内容，以智能识别并提取程序源代码中方法签名、参数类型、功能模块等关键要素，智能体再将这些焦点信息系统地处理和组织输出为生成提示中，为生成测试代码提供明确而充分的指导信息。因此，我们在传统程序理解方法上同步引入了 LLM 参与解析代码，确保程序理解智能体能够精准映射出待测源代码的真实意图。

2.2 测试生成

在此过程中，提示信息将不再局限于简单的任务描述，而是整合了程序理解智能体带来的更加丰富和完整的过程信号。此时，生成提示不仅将明确指定测试对象，而且还会包含对象参数、测试任务、功能描述、生成标准、测试骨架等测试要求。这些结构化信息创建的生成提示，将有效地辅佐代码生成智能体按照既定的测试需求和规范，生成符合质量要求的测试代码。由程序理解智能体增强后的提示将显著提升测试代码生成的准确性，而且可以确保输出测试能够更好地满足测试意图。

2.3 测试验证

与传统人工校验或系统脚本验证方式不同，我们的方法将通过结构化设计提示，设计包含全面而细致的验证标准去指导智能体参与测试验证过程，利用其强大的自然语言处理和代码理解能力，自动分析和验证测试代码的语义正确性、逻辑完整性和测试可行性，实现对测试代码的全面验证。让智能体参与测试验证过程不仅要能确保测试代码符合语法规则、测试逻辑符合预期、最终测试可执行编译且没有语法语义错误，还要能大幅提高测试效率，发现传统方法难以察觉的潜在问题。这种让智能体验证测试的方法对于保障生成测

试质量和可行具有重大意义。

2.4 修复错误

针对包含错误的单元测试代码，我们基于目标修复策略实现，策略共有三个等级。智能体输出验证结果时会自动标记每一个错误对应地修复等级，并根据修复等级自动完成错误代码修复，若此时修复仍失败，智能体会自动升级修复等级，并记录错误报告。若出现新类别，智能体将根据错误的影响范围划定选择修复等级，修复成功时，智能体将自动记录标签并归类。当前修复规则，是我们根据修复主题的文献和 demo 实验的结果制定，我们将其中常见的错误类型和解决方案进行类别定义，使智能体能够快速的选择更佳的修复方式，并高效地处理常见的错误类型，直到修复所有的错误或轮次结束。基于 demo 实验结果的整理和学习，我们预设整个修复过程将在三轮内完成，且修复完成后都必须再次验证。

3. 集成智能体与多智能体协作实现

本研究的核心原理是利用 LLM 对代码的语义理解、逻辑推理和生成能力，结合智能体的记忆能力、规划能力和行动能力，再结合具体单元测试生成，通过多智能体协作，构建一种自动化、智能化的单元测试代码生成方法。这种多智能体协作方法，通过整合由 LLM 驱动的多个智能体进行程序理解、代码生成和测试验证，从而提升单元测试代码生成自动化程度与代码质量。我们根据单元测试全过程的划分将测试代码生成同步划分为三个阶段，如图 3-2 所示：

3.1 程序理解智能体

此阶段的智能体负责深入理解待测试代码的语义和逻辑结构。分析代码的语法、依赖关系和执行路径，提取关键信息，如方法签名、参数类型、预期行为等。此外，程序理解智能体还能识别代码中的潜在问题和测试点，为后续的代码生成和测试验证提供基础。

3.2 代码生成智能体

在程序理解的基础上，代码生成智能体利用 LLM 的能力和程序理解智能体输出的生成提示，自动识别提取的测试点和预期行为，生成单元测试代码。这个阶段的智能体具备良好的代码生成能力，能够生成结构合理、逻辑清晰、覆盖率高的测试代码。同时，生成的测试代码能更好地处理复杂的测试场景，如边界条件、异常处理等。

3.3 测试验证智能体

生成的测试代码需要通过测试验证智能体的严格检验。这个阶段的智能体负责模拟测试代码的动态执行，检查其是否能够正确运行，是否能够达到预期的测试覆盖率，以及是否能够发现潜在的代码缺陷。测试验证智能体还具备一定的错误检测和修复能力，当发现测试失败时，能够准确定位问题所在，并提供详细的错误信息和解决方案。

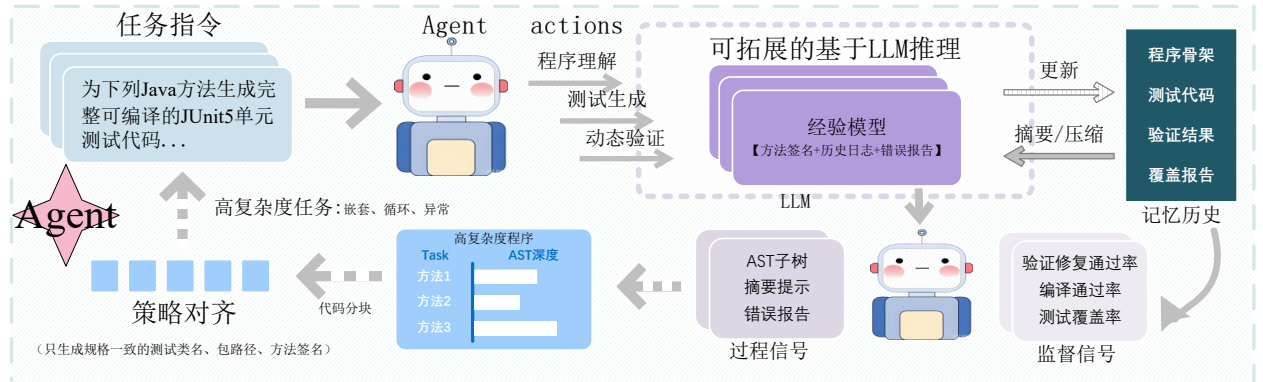


图 3-3 多智能体驱动单元测试全过程流程概述

通过这三个阶段的紧密协作，多智能体系统能够实现从程序理解到测试代码生成，再到测试验证的全自动化流程。每个智能体在完成其特定任务的同时，还能够根据反馈信息进行自我学习和优化，从而不断提高整个系统的自动化程度和代码质量。这种基于 LLM 的多智能体协作单元测试代码生成方法，不仅能够大幅提高测试代码的生成效率，还能够确保测试代码的高质量和高可靠性，为软件开发和维护提供强有力的支持。

四、本研究课题可能的创新之处（不少于 500 字）

本研究聚焦于让 LLM 全面参与单元测试全过程这一核心原理，目的在以智能体协作解决半自动化问题，为后续的研究奠定了坚实基础。在软件开发流程中，单元测试是保障代码质量的关键环节，传统的单元测试往往依赖人工编写测试代码或者人工辅助生成测试代码，效率较低且容易出现人为错误。而本研究提出构建一种多智能体协作的单元测试代码自动生成方法，该方法能够紧密跟随单元测试的全部过程，由智能体提供精准指导，科学决策，全自动生成单元测试。这一方法旨在将单元测试生成的所有环节都交由三个智能体负责。各智能体协同合作，深度理解并融入到单元测试的环境中，如同照镜子一般，能够清晰、准确地明白单元测试的各项需求和目标。本文存在以下创新：

1. 动态验证方法

本研究提出的动态验证方法，是一种在测试代码生成过程中实时进行验证的机制。这种方法不再依赖于传统的“一刀切”编译方式，而是对测试代码进行逐行或逐段的实时验证。这样可以及时发现并修正错误，避免无效或错误的测试代码的生成，从而提高测试代码的质量和覆盖率。动态验证方法还包括对测试代码的执行逻辑进行模拟，以确保测试代码能够覆盖正常流程、边界条件以及异常处理等关键场景。这种方法能够确保测试代码不仅在语法上正确，而且在逻辑上也符合预期。

2. 目标修复策略规则

目标修复策略是一种智能化的错误修复机制，它根据错误的性质和影响范围，自动选择最合适的修复策略，如图 4-1 所示。这种策略能够针对代码行错误、方法体错误或整个类错误进行精确修复，从而提高修复的准确性和效率。修复策略规则还能够根据历史修复经验进行自我学习和优化，使得智能体在未来遇到类似问题时能够更快、更准确地进行修复。这种自我优化的能力使得测试代码生成过程更加智能化和自动化。

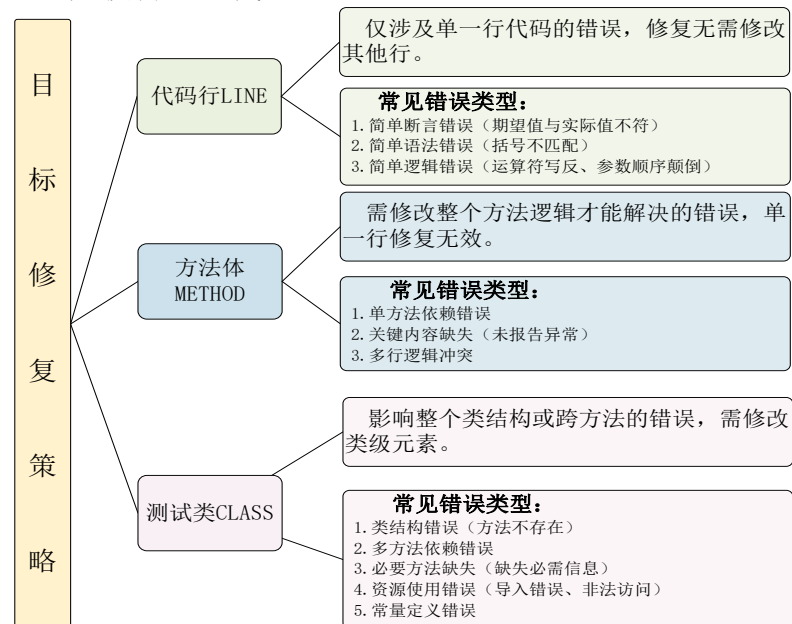


图 4-1 目标修复策略

3. 基于 LLM 多智能体协作的单元测试代码生成方法

本研究提出的基于 LLM 多智能体协作的单元测试代码生成方法，是一种测试代码生成范式。这种方法通过整合由 LLM 驱动多个智能体进行程序理解、代码生成和测试验证，从而提升单元测试代码生成的自动化程度与代码质量。多智能体协作方法使得每个智能体都能够专注于其最擅长的任务，如程序理解智能体负责深入理解待测试代码，代码生成智能体负责自动生成测试代码，测试验证智能体负责验证和修复测试代码。这种分工合作的方式不仅提高了测试代码生成的效率，还确保了测试代码的高质量和高可靠性。此外，多智能体协作方法还具备自我学习和自我优化的能力，能够根据测试过程中的反馈和结果不断优化自身的测试策略和方法，从而实现更高效、更可靠的单元测试代码生成,如图 4-2 所示。

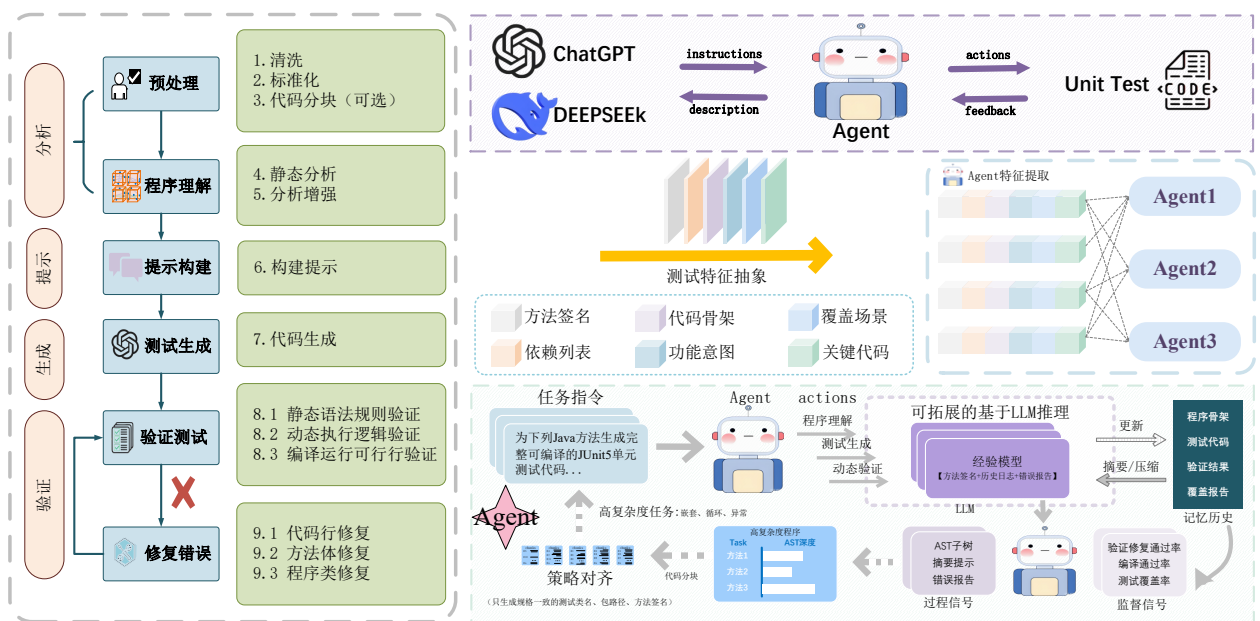


图 4-2 基于 LLM 多智能体协作的单元测试代码生成方法概述

通过这些创新点，本研究提出的单元测试代码自动生成方法能够显著提高测试代码生成的自动化程度和效率，同时确保测试代码的高质量和高可靠性，为软件开发和维护提供强有力的支持。

五、研究基础与工作条件（1. 与本项目相关的研究工作积累基础 2. 包括已具备的实验条件，尚缺少的实验条件和拟解决途径）（不少于 500 字）

1. 数据基础

我们从 openrouter 官网选择 GPT-4o-mini 和 DeepSeek-R1 这两个大模型，为了确保实验的公平性与可比性，将严格遵循“零训练干预”原则，对两个模型不进行任何数据集的针对性训练。评估指标采用编译通过率与测试覆盖率作为衡量标准。编译通过率由系统最终能够成功编译运行的测试类数量与所有待测试类总数之比计算得出。并在终端执行 `mvn test` 命令运行所有可编译测试类，借助 JaCoCo 代码覆盖率收集工具，系统采集指令覆盖率、分支覆盖率、行覆盖率、方法覆盖率及类覆盖率等数据，对测试代码以及基础模型进行评估验证其有效性。

2. 理论基础

针对 LLM 驱动自动化单元测试生成在理解测试意图方面的准确性和重复提示的幻觉问题，我们展开了深入研究。重点回顾了过去三年中和此相关的论文，包括但不限于基于程序理解、错误修复和提示优化主题。针对这些文献的研究方法和原理实现进行了深入剖析，并结合自动化单元测试的核心概念，我们初步提出让 LLM 参与自动化单元测试全过程这一理念，初步构建单元测试全过程的概念框架。并基于框架内容和研究问题初步设计了适用于各阶段的提示内容。通过这些文献的研究，我们旨在创建一种基于 LLM 的多智能体协作单元测试生成方法，减少 LLM 幻觉问题，提高测试意图理解的准确度，从而提升单元测试生成的质量和效率。

3. 实验基础

统一实验环境配置，包括但不限于模型调用参数、硬件资源（均部署于 NVIDIA GeForce GTX 1650）以及运行平台 IntelliJ IDEA 2019.2.3）。目前，我们基于单元测试全过程这一概念和原理搭建了一个简单的基础模型框架，基于 GPT-4o-mini 和 DeepSeek-R1 这两个模型并选择 Defects4J 数据集的 Commons-CLI 项目进行了初步实验。实验数据如表 5-1 所示，单元测试各阶段方法对测试代码生成重要性如表 5-2 所示，部分实验结果如表 5-3 所示。结果显示我们的方法对于提升测试覆盖率有着很好的趋势。

表 5-1 实验数据

Commons CLI	总待测试类	总待测试方法	圈复杂度	有效代码行数
cli	29	405	817	1448
help	15	143	242	506
all	44	548	1059	1954

表 5-2 不同模块驱动测试代码生成实验结果

模块名称	总待测试类	总待测试方法	圈复杂度	有效代码行数	测试覆盖率
程序理解	39	452	781	1570	75%
验证+修复	35	399	690	1005	85%

表 5-3 实验结果

方法	模型	对象	指令覆盖率	分支覆盖率	行覆盖率	方法覆盖率	类覆盖率
our	gpt-4o-mini	cli	35%	23%	40%	47%	97%
		help	41%	30%	44%	53%	100%
	deepseek-r1	cli	29%	19%	35%	43%	90%
		help	33%	15%	38%	55%	100%
chatunitest	gpt-4o-mini	cli	42%	34%	45%	57%	66%
		help	27%	27%	29%	26%	60%
	deepseek-r1	cli	2%	0%	3%	2%	14%
		help	2%	0%	3%	2%	13%
hunman	—	cli	98%	95%	98%	98%	100%
		help	99%	97%	99%	97%	100%

学位论文工作计划

时间	研究内容	预期效果
2025-10 – 2026-01	设计并实现基础模型	成功有效地通过 demo 项目测试
2026-01 – 2026-04	处理所有数据集，设计并实现单元测试全过程方法并部署至基础模型	平均所有数据集在评估指标上相持于 chatunitest
2026-04 – 2026-07	基础模型调试并集成智能体，在实验中训练多智能体经验学习与自主决策能力	平均所有数据集在评估指标上优于 chatunitest
2026-07 – 2026-10	总结前期工作，准备中期答辩	完成中期答辩
2026-10 – 2027-01	完成相关工作整理，撰写毕业论文	完成毕业论文撰写
2027-02 – 2027-06	论文送审及修改，毕业答辩	完成毕业答辩

导师意见：

导师（签名）：

日期： 年 月 日

评 定 小 组 成 员	姓 名	职 称	单位名称	职务

开题报告小组意见：

组长（签名）：

日期： 年 月 日

学院意见（签章）：

负责人：

日期： 年 月 日